Trabajo Práctico: Motor de búsqueda de funciones similares vía AST

1. Objetivo

Diseñar e implementar una herramienta que, dado un conjunto de archivos fuente en Python, extraiga todas las funciones definidas, las represente mediante su Árbol de Sintaxis Abstracta (AST) normalizado, y permita buscar qué funciones del conjunto son estructuralmente similares a una función dada — replicando, a escala reducida, el principio detrás de herramientas reales de detección de código duplicado (SonarQube, PMD/CPD) o de similitud entre entregas (MOSS, JPlag).

2. Marco conceptual

El AST que produce el propio intérprete de Python al parsear un programa es, ni más ni menos, el árbol de derivación que la materia estudia de forma abstracta: el resultado de aplicarle la gramática del lenguaje a una cadena de entrada. Comparar código por similitud estructural (en lugar de por texto plano) es comparar árboles de derivación, no strings — la misma idea conceptual que subyace a la traducción dirigida por la sintaxis vista en el curso.

Se recomienda explorar previamente el módulo ast de la biblioteca estándar (ast.parse, ast.walk, ast.NodeVisitor, ast.dump, ast.unparse) antes de comenzar el diseño.

3. Alcance del trabajo

- El universo de búsqueda es un repositorio o carpeta local con una cantidad moderada de archivos .py (decenas de archivos, no miles).
- La comparación es entre funciones individuales (no entre archivos completos ni entre clases).
- La similitud se mide sobre una secuencia de tokens obtenida de un recorrido pre-order del AST normalizado.
- Comparación todos-contra-todos ($O(n^2)$) es aceptable como estrategia de búsqueda.

4. Requisitos funcionales

4.1 Extracción

- Recorrer recursivamente un directorio dado, parsear cada archivo .py con ast.parse().
- Extraer todas las FunctionDef (y opcionalmente AsyncFunctionDef), registrando: nombre original, archivo, línea y su subárbol AST.

4.2 Normalización

- Reemplazar identificadores de variables y parámetros por tokens genéricos (VAR1, VAR2, ...) preservando la correspondencia dentro de una misma función.
- Reemplazar literales (números, strings) por un token genérico (LIT), salvo que el grupo justifique lo contrario.
- El nombre de la propia función NO debe influir en la comparación.

4.3 Serialización y métrica de similitud

- Convertir el AST normalizado en una secuencia lineal de tokens (tipo de nodo + tokens normalizados) mediante un recorrido pre-order.
- Implementar al menos una métrica de similitud entre dos secuencias: similitud de Jaccard sobre n-gramas (n a elección, justificar) y/o distancia de edición (Levenshtein) normalizada.
- Definir y documentar un umbral de similitud a partir del cual dos funciones se consideran 'candidatas a duplicado'.

4.4 Motor de búsqueda / interfaz

- Dada una función objetivo (por archivo + nombre, o pegando el código), devolver el top-K (configurable) de funciones más similares del repositorio, con su score.
- Interfaz de línea de comandos (CLI) simple, por ejemplo:

```
python buscar_similares.py --dir ./mi_repo --funcion archivo.py:mi_funcion
--top 5
```

5. Entregables

- Código fuente completo, organizado en módulos (extracción / normalización / similitud / CLI).
- Suite de tests (pytest u otro) con al menos: 2 casos positivos (funciones clonadas con variables renombradas o líneas reordenadas) y 2 casos negativos (funciones claramente distintas).
- Informe breve (máx. 2 páginas) que explique: decisiones de diseño, métrica elegida y por qué, limitaciones conocidas (qué tipo de clones NO detecta la herramienta) y posibles mejoras.
- Repositorio git con historial de commits que refleje el avance incremental del trabajo.

6. Modalidad

- Grupos de entre 2 y 3 integrantes.
- Todos los elementos de la entrega deben estar subidos al repositorio. En caso de optar por un repositorio privado deberán darnos acceso previamente.
- Fecha límite de entrega: viernes 30/10 23:59 hs.
- Defensa oral individual breve (4/11): cada integrante debe poder explicar y modificar en vivo cualquier parte del código.

Recursos sugeridos

- Documentación oficial del módulo ast: https://docs.python.org/3/library/ast.html
- ast.NodeVisitor y ast.NodeTransformer para recorrer/transformar árboles.
- ast.unparse() (Python ≥ 3.9) para regenerar código a partir de un AST, útil para debugging.
- Material de la cátedra sobre gramáticas y árboles de derivación.